WinContextDeploy

Post-image configuration and quick device diagnostics

Technician's manual

What the tool does, what it leaves to you,
and how to extend it

Built with Typst from `docs/manual.typ` · MIT licence

Contents

CHAPTER 1

Introduction

1.1 Why this tool exists

A freshly imaged Windows machine is not a finished machine. The image is the same for everyone, but the machine in front of you is not: it has a battery or it does not, it runs its applications locally or in a remote session, it belongs to a technician who needs a French interface or an English one.

Closing that gap by hand takes twenty minutes of clicking through the same settings dialogs, in the same order, and it takes them again on the next machine. Twenty minutes is not the problem. The problem is that the twentieth machine of the week gets a slightly different treatment from the first, and nobody can say afterwards which settings were actually applied.

WinContextDeploy is the answer to that: a technician runs it once, answers four or five questions, and it applies the settings that depend on the machine's context, opens the applications that need to be checked by eye, and prints a checklist of what was done, what was skipped, and what still has to be done by hand.

> **CAUTION**
> It does **not** install software. Applications are expected to arrive through your imaging pipeline. This tool confirms they are there and puts the ones a technician must eyeball in front of them.

1.2 The three ways to launch it

There is one launcher, `WinContextDeploy.cmd`, and it covers three situations.

`WinContextDeploy.cmd`	Run **in place** — correct for a `git clone`
`WinContextDeploy.cmd -Usb`	Copy to `%TEMP%`, run there, write the log back
`powershell -File src\Invoke-WcdConfiguration.ps1`	Direct, scriptable

Run in place is the default and the one to use after a `git clone`. Nothing is copied and nothing is deleted.

`-Usb` is for a USB key. PowerShell off removable media is slow and the key can be pulled mid-run, so the project is copied to a uniquely named folder under `%TEMP%`, run from there, and the run's history block is appended back to `log.txt` next to the launcher before the copy is removed.

Direct is how you drive it from another script: every prompt has a matching parameter, so `-NonInteractive` runs the whole thing without asking anything.

Add `-FR` or `-EN` to either `.cmd` form to force the interface language. Without one, it follows the system locale.

1.3 What this manual covers

- **Chapters 2 and 3** — how to run the tool, and what it does on its own.
- **Chapter 4** — what it deliberately leaves to you.
- **Chapter 5** — how the project is put together, in diagrams.
- **Chapter 6** — PowerShell, explained from nothing.
- **Chapter 7** — adding your own module.

- **Chapter 8** — reading the diagnostic and the logs when something goes wrong.

CHAPTER 2

Quick start

2.1 Before you run it

Windows	Windows 10 or 11.
PowerShell	5.1 or later. Windows ships with it; nothing to install.
Administrator	Only the power settings need it. The tool offers the UAC prompt itself and carries on perfectly well if you decline.
The manifest	Open `WinContextDeploy.psd1` and point it at the applications, printers and URLs of **your** environment. The shipped entries are generic examples.

> **CAUTION**
> Read the manifest before the first run on a real machine. The tool changes system settings: the taskbar, the regional separators, the power plan and the keyboard layout.

2.2 Running it

Double-click `WinContextDeploy.cmd`. A console window opens, the banner is drawn, and the questions start.

```
 __          ___       _____           _          _  _____           _
 \ \        / (_)     / ____|         | |        | || __ \         | |
  \ \  /\  / / _ _ __ | |    ___  _ __ | |_ _____ _| || |  | | ___ _ __ | | ___  _   _
   \ \/  \/ / | | '_ \| |   / _ \| '_ \| __/ _ \ \/ / || |  | |/ _ \ '_ \| |/ _ \| | | |
    \  /\  /  | | | | | |___| (_) | | | | ||  __/>  <| || |__| |  __/ |_) | | (_) | |_| |
     \/  \/   |_|_| |_|\_____\___/|_| |_|\__\___/_/\_\\__|_____/ \___| .__/|_|\___/ \__, |
                                                                     | |           __/ |
                                                                     |_|          |___/

Post-image configuration and quick device diagnostics
```

Every question works the same way: **Left** and **Right** move between the choices, **Enter** confirms. The highlighted option is the default, so holding Enter through all of them is a valid answer.

2.3 The menu questions

2.3.1 Windows system language

```
Windows system language
  fr-CA = French Canadian Windows interface  |  en-US = American English
Use Left/Right to change, then Enter to confirm.
 FR: fr-CA    EN: en-US
```

Sets the display language, the keyboard layout, the user culture and the home region. This is the language **Windows** will speak, and it is independent of the language the tool itself uses for its prompts.

2.3.2 Machine form factor

```
Machine form factor
  Laptop = has a battery and a lid  |  Desktop = neither
```

```
  Selects the power profile: battery and lid-close settings.
 L: Laptop    D: Desktop
```

On a **Laptop**, the tool sets the screen timeout on battery, the screen timeout on AC, and makes closing the lid do nothing. On a **Desktop** the battery and lid steps do not apply, and the checklist says so rather than leaving them out.

2.3.3 Machine environment

```
Machine environment
  Workstation = full local machine  -> checks the locally installed applications
  Citrix / VDI = thin endpoint      -> its applications live in the remote session
 W: Workstation    V: Citrix / VDI
```

This is what decides which applications are relevant. On a VDI endpoint the line-of-business applications live in the remote session, so checking for them locally would report a problem that is not one — the checklist marks them **Not Applicable** instead.

2.3.4 Open the configuration applications

```
Open Workstation configuration applications?
  Opens the Application Targets declared in WinContextDeploy.psd1.
  Answer Yes unless applications were already opened manually.
 Y: Yes    N: No
```

Answer **No** if you have already opened everything by hand. The applications then appear on the checklist as manual steps rather than silently disappearing from it.

2.3.5 Engineering workstation

```
Engineering workstation? (offers the optional tools)
  Offers the extras declared Prompt in WinContextDeploy.psd1.
  Answer No for a standard machine.
Y: Yes   N: No
```

Answering **Yes** opens a numbered menu of the optional tools, where several can be picked at once by typing their numbers together — `13` selects the first and the third.

> **NOTE**
> This question only appears when the manifest declares at least one entry with `Prompt = $true`. With none, the tool skips it entirely.

2.4 The result: the final diagnostic

When every module has run, the tool prints its report twice. First grouped by module, which is the view that tells you whether the run itself went well:

```
================================================
         FINAL DIAGNOSTIC - BY MODULE
================================================
  [x]  Config-Power              OK        5 step(s)
  [x]  Config-Decimal            OK        1 step(s)
  [x]  Config-TaskbarLeft        OK        2 step(s)
```

```
  [x]  Config-Language           OK        2 step(s)
  [!]  Config-Applications       WARNING   9 step(s)  Warnings: AppVpn
  [x]  Config-DeviceManager      OK        1 step(s)
  [x]  Config-Disk               OK        2 step(s)
  [x]  Config-Network            OK        3 step(s)
```

Then as a flat checklist, which is the view you actually work from — one line per thing that had to happen on this machine:

```
==============================================
         FINAL DIAGNOSTIC - BY STEP
==============================================
  [x]  Taskbar aligned left     OK
  [x]  Display language         OK
  [x]  Keyboard layout          OK
  [x]  Decimal separator        OK
  [x]  Power options            OK
  [x]  Device Manager           OK
  [x]  Disk health              OK
  [x]  Free space               OK         412 GB free of 476 GB
  [x]  Network adapters         OK
  [x]  Software Center          OK
  [!]  VPN client               WARNING    No matching process running (PanGPA, PanGPS).
                                           -> Confirm VPN client is installed and started,
                                              or mark the entry Optional.
  [!]  ERP client               ERROR      ERP client not found at C:\ProgramData\...
                                           -> Update Applications['ERP client'].Target in
                                              WinContextDeploy.psd1, or remove the entry.
  [-]  Citrix Workspace         N/A        Not applicable to the chosen Environment.
  [-]  Mail signature           MANUAL     Must be done manually.
  [-]  Wi-Fi                    MANUAL     Must be done manually.
  [-]  Network drives           MANUAL     Must be done manually.

Summary: 8 OK, 1 warning(s), 1 error(s), 7 manual, 1 N/A.
```

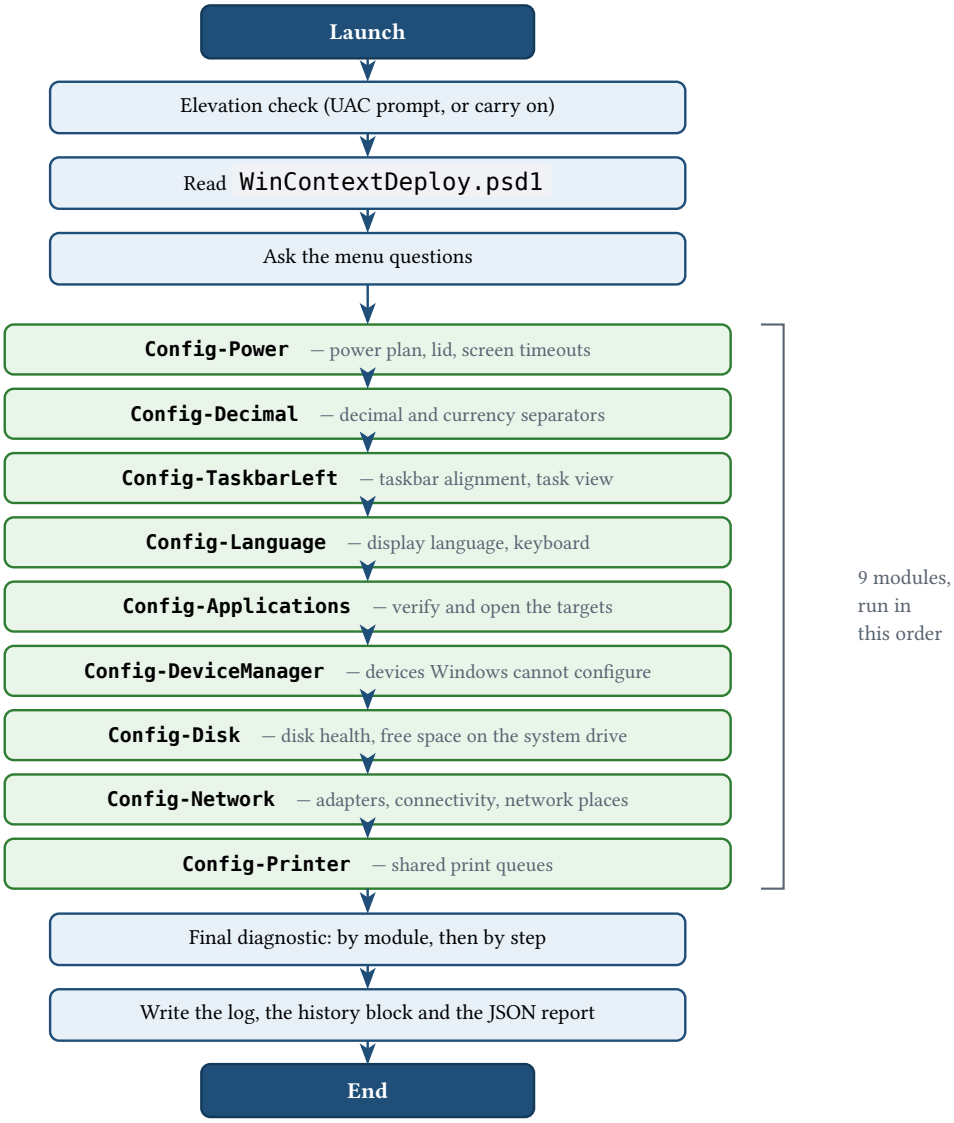
The arrow lines are the point of the whole report. A failure that only says *"the specified file was not found"* tells a technician nothing; a failure that names the manifest key to edit tells a technician exactly what to do next.

CHAPTER 3

What the tool does automatically

3.1 The full execution flow

From the double-click to the final report, the run always follows the same path. Your answers change what each module **does**, never the order they run in.



> **NOTE**
> A module that crashes does not stop the run. Its failure is recorded, the next module starts, and you still get a complete report at the end. The only thing a technician cannot use is a report that never appears.

3.2 What each module does

3.2.1 Config-Power — power plan, lid and screen timeouts

Sets the screen timeout to 10 minutes on battery and 15 on AC, makes closing the lid do nothing, and applies the active power scheme. On a `Desktop` the battery and lid steps do not apply.

This is the **only** module that needs Administrator. Without it, the module attempts nothing and every step reports:

```
  [!]  Power options           WARNING    Screen timeout on AC: powercfg requires
                                          Administrator.
                                          -> Requires Administrator. Relaunch elevated
                                             to apply.
```

> **TIP**
> The sleep steps (`standby-timeout-*`) are deliberately switched off: they are blocked by Group Policy in most managed environments, so attempting them only produces noise. The commented rows in the module's step table are what to restore if that is not true for you.

3.2.2 Config-Decimal — decimal and currency separators

Forces the period as the decimal separator and the comma as the thousands separator, for both numbers and money. A French-Canadian Windows uses a comma for decimals by default, which quietly corrupts anything pasted into a spreadsheet built around the other convention.

3.2.3 Config-TaskbarLeft — taskbar alignment and task view

Aligns the taskbar to the left and hides the Task View button — Windows 11 defaults that most users on a fleet want changed once, not every time.

3.2.4 Config-Language — display language and keyboard

Applies the display language, the keyboard layout, the user culture, the system locale and the home region for the culture you chose.

> **NOTE**
> The system locale and the home region need Administrator. Without it, the module applies everything else and notes the two it could not do. Some Windows Store applications also only pick up the new language after a sign-out.

3.2.5 Config-Applications — verify and open the targets

The manifest-driven core of the tool. It walks the `Applications` list in `WinContextDeploy.psd1` in order and runs each entry according to its `Action` :

`Launch`	Starts an application: a `.lnk` , an `.exe` , or a command on `PATH` .
`OpenFolder`	Opens a folder in Explorer.
`OpenUrl`	Opens a URL in the default browser.
`CheckProcess`	Verifies a process is running. Launches nothing. `Target` is a list of process names, and any one of them counts.
`CheckPath`	Verifies a path exists. Launches nothing.

Adding, removing or reordering an application is a manifest edit. No code changes, no new module.

3.2.6 Config-DeviceManager — devices Windows cannot configure

Reads every Plug-and-Play device and reports the ones with a problem code. A missing driver on a freshly imaged machine is exactly what this catches, before the user finds it a week later.

Codes that resolve themselves — a pending reboot, a device not currently connected — are warnings. A missing or broken driver is an error.

3.2.7 Config-Disk — disk health and free space

A freshly imaged machine can still be sitting on a dying drive, or on a partition far smaller than the image expects. Neither shows up anywhere else in the run, and the technician usually finds out weeks later, from the user.

Two steps. `Disk health` reads the health each disk reports: `Healthy` passes, `Warning` is a warning, `Unhealthy` is an error that names the drive and its serial number so the right one gets pulled. A status the drive does not report at all is a warning rather than a silent pass — an unreadable disk is not a healthy one. `Free space` reads the system drive and reports the figure either way, so an OK row still tells you how much room the machine has.

Removable disks are left out of the health check. A technician's USB key is not the machine's disk, and it must never block a handover. Free space is read on the system drive only, for the same reason.

> **TIP**
> How much free space is enough is a fleet decision, so it lives in the manifest:
>
> ```
> Disk = @{
>     MinFreeGB = 20
> }
> ```
>
> Omit the key and the threshold is 20 GB. Below it the checklist warns and names the figure and the threshold; the remediation points back at `Disk.MinFreeGB`, so a threshold that is wrong for your fleet is one edit away.

> **NOTE**
> Wear percentage and drive temperature — `Get-StorageReliabilityCounter` — are deliberately not read. They are genuinely useful on refurbished machines, but they need Administrator and return nothing on plenty of consumer SATA and NVMe drives, so the step would silently report nothing on a large slice of any fleet.

3.2.8 Config-Network — adapters, connectivity and network places

Inventories the active adapters (the virtual ones — Bluetooth, loopback, VPN, hypervisor — are filtered out), pings the configured target **from the Wi-Fi adapter**, and triggers the user's "Refresh My Network Places" shortcut.

> **TIP**
> The ping source address is forced to the Wi-Fi adapter on purpose. With a cable plugged in, an ordinary ping would be answered over Ethernet and pass while the Wi-Fi is quietly broken.
>
> A blocked ping is a warning, never a hard failure — many corporate networks drop ICMP to the internet. Point `Network.PingTarget` at your gateway or an internal host if that is your case.

3.2.9 Config-Printer — shared print queues

Connects the shared print-server queues listed in the manifest's `Printers` array, using the built-in `Add-Printer` cmdlet. Already connected is a no-op, so running the tool twice is safe, and an unreachable print server is a warning rather than a crash.

Leave `Printers = @()` and the module is skipped entirely; printers then stay on the checklist as a manual step.

CHAPTER 4

What stays manual

Some things are deliberately not automated. Automating them badly is worse than not automating them: a step that half-works on some machines is a step nobody can trust on any machine.

They appear on the checklist anyway, marked `MANUAL`, so a technician cannot finish a machine having forgotten one.

Mail signature	Depends on the person, not the machine: their name, title and telephone number. There is nothing about the machine that can supply them.
Wi-Fi	Joining a network usually needs a credential or a certificate the tool has no business handling.
Network drives	Which shares a user gets is a directory decision. `Config-Network` refreshes network places, but it will not invent mappings.
Account sync	Signing into the account is the user's action, at their first login.
Windows desktop	Icon layout and personal shortcuts — a preference, not a configuration.
Browser favourites	Imported from the user's profile or pushed by policy, not by a local script.
Printers	Manual **only when the manifest declares none**. Fill in the `Printers` array and this becomes automatic.

> NOTE
> A manual step is not a failure and not a skipped step. The tool distinguishes three things that look alike:
>
> - **MANUAL** — deliberately yours to do.
> - **N/A** — does not apply to this machine's form factor or environment, so its absence is correct.
> - **ERROR** — should have worked, and did not.

CHAPTER 5

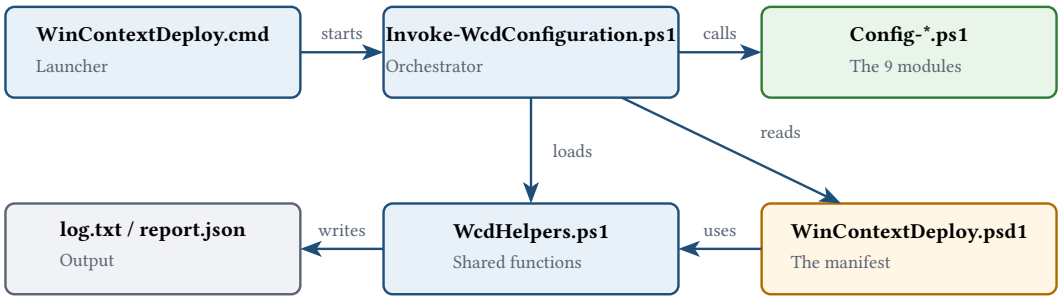
How the project is put together

5.1 C4 diagram — the architecture

A C4 diagram describes a system at progressively closer zoom levels. Think of it as a map: level 1 is the satellite view of who uses what, level 2 shows the buildings inside.

5.1.1 Level 1 — context: who interacts with what

5.1.2 Level 2 — containers: the pieces inside



> **TIP**
> Notice which way the arrows point. The orchestrator reads the manifest and hands each module exactly what it needs; the modules return a Result and write to the log. Nothing reads back out of a module. That is what makes each one testable on its own — and why adding a module changes nothing else.

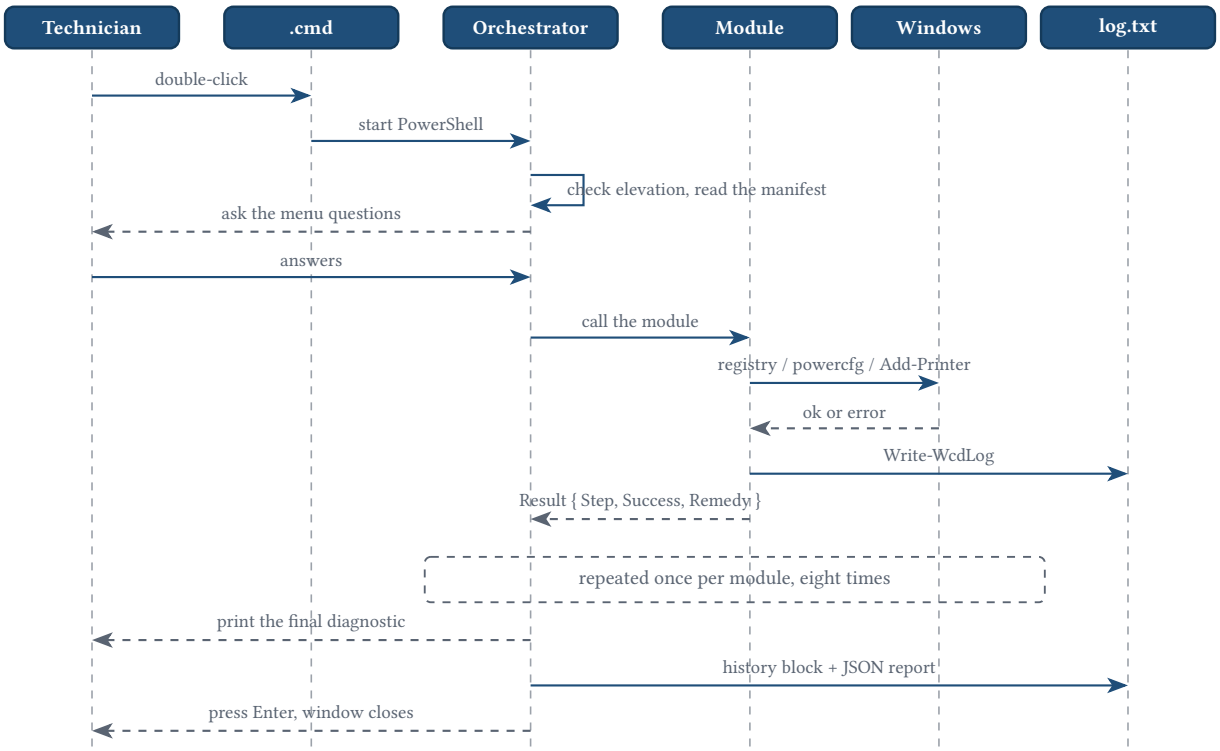
5.2 The folder tree

```
WinContextDeploy/
├── banner.txt                      startup logo, editable
├── WinContextDeploy.cmd            the one launcher
├── WinContextDeploy.psd1           the manifest — edit this
├── src/                            the code
│   ├── Invoke-WcdConfiguration.ps1 orchestrator
│   ├── WcdHelpers.ps1              shared functions
│   └── Config-*.ps1                one file per module
├── tests/                          Pester tests, one file per module
└── docs/                           this manual
```

`WinContextDeploy.psd1`	Every environment-specific value: application paths, printers, URLs, the ping target. This is the file you edit; the rest is the same everywhere.
`src/`	One file per module, plus the orchestrator and the shared helpers. Adding a module means adding a file here.
`tests/`	One Pester test file per module. A new module gets a new test file.
`docs/`	This manual, as Typst source and as a built PDF.

5.3 Sequence diagram — how the pieces talk

This is one run, in the order things actually happen.



> **NOTE**
> The module never touches the screen. It returns a Result and raises progress events; the orchestrator decides what that looks like. That is exactly why a module can be tested with no console attached at all.

CHAPTER 6

PowerShell, explained simply

You do not need to be a developer to extend this tool. You need four ideas, and this chapter covers all four with examples taken from the code you already have.

6.1 What PowerShell is

Windows has a graphical interface and a text one. Clicking through the Settings app is fine for one machine. It is unbearable for twelve a week, and — the part that matters — it leaves no record of what was actually done.

PowerShell is the text interface. You write down the steps once, in a file, and Windows performs them the same way every time. That file **is** the record.

A PowerShell file has the extension `.ps1`. That is all a "script" is.

6.2 The four building blocks

6.2.1 Variables — remember a value

A variable is a labelled box. It starts with `$`, you put something in it, and you use the label afterwards instead of repeating the value.

```
# Store the registry key we are about to change
$intlPath = 'HKCU:\Control Panel\International'

# Use it twice without retyping it
Set-WcdRegistryValue -Path $intlPath -Name 'sDecimal'      -Value '.'
Set-WcdRegistryValue -Path $intlPath -Name 'sThousandSep'  -Value ','
```

If the key ever moves, you change one line, not two. On a longer script, not twenty.

6.2.2 Conditions — do something only sometimes

`if` runs a block only when something is true. This is how the tool adapts to the machine in front of it, and it is the whole idea behind the Form Factor question:

```
if ($FormFactor -eq 'Laptop') {
    # Only a laptop has a lid to close
    Invoke-WcdPowerCfg '/setacvalueindex' 'SCHEME_CURRENT' 'SUB_BUTTONS' 'LIDACTION' '0'
}
```

`-eq` means "equals". The others read the same way: `-ne` not equal, `-gt` greater than, `-lt` less than, `-match` matches a pattern.

6.2.3 Functions — name a piece of work

A function is a named block you can call from anywhere. Every module in this project is a function.

```
function Set-WcdDecimalConfiguration {
    param(
        [string]$LogPath        # what the caller passes in
    )

    # ... the work ...
```

```
    return [pscustomobject]@{ Step = 'DecimalAndCurrency'; Success = $true }
}

# Calling it
$result = Set-WcdDecimalConfiguration -LogPath 'C:\temp\log.txt'
```

`param(...)` declares what the caller may pass. `return` hands something back.

6.2.4 try / catch — the safety net

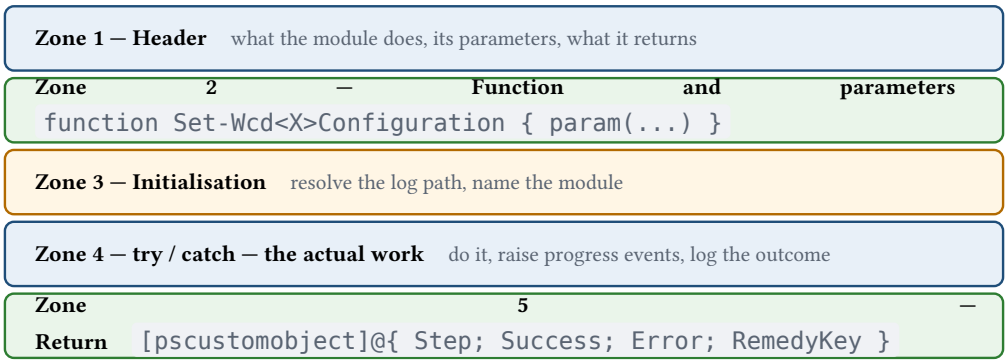
This is the one that makes the whole tool usable. `try` attempts something; `catch` runs only if it failed, instead of the whole script stopping.

```
try {
    Set-WcdRegistryValue -Path $intlPath -Name 'sDecimal' -Value '.'
    $success = $true
} catch {
    # $_ is whatever went wrong
    $note = $_.Exception.Message
    $success = $false
}
```

> **TIP**
> This is why one locked registry key does not cost you the whole run. The module records the failure, returns, and the next module starts. A technician gets a full report of a partly successful run — which is far more useful than a script that stopped at step three with a red wall of text.

6.3 The anatomy of a module

`Config-Decimal` is the simplest module in the project: one step, no dependencies. Every other module has the same five zones — only zone 4 differs.



Here is the real module, with the zones marked:

```
# ------------------------------------------------------------- ZONE 1
# Config-Decimal.ps1 - forces the decimal and currency separators to a period.
# Entry point: Set-WcdDecimalConfiguration. Requires WcdHelpers.ps1.

function Set-WcdDecimalConfiguration {
    [CmdletBinding()]
```

```
param(                                                          # ZONE 2
    [string]$LogPath,                 # where to write the log
    [scriptblock]$ProgressCallback    # how to report progress
)

# ------------------------------------------------------------ ZONE 3
$resolvedLogPath = Resolve-WcdLogPath -CandidatePath $LogPath
$intlPath = 'HKCU:\Control Panel\International'
$moduleName = 'Config-Decimal'

# ------------------------------------------------------------ ZONE 4
try {
    # Tell the orchestrator this step is starting
    Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
        -ModuleName $moduleName -StepKey 'DecimalAndCurrency' -Event 'Start'

    # The actual work
    Set-WcdRegistryValue -Path $intlPath -Name 'sDecimal'       -Value '.'
    Set-WcdRegistryValue -Path $intlPath -Name 'sThousandSep'   -Value ','
    Set-WcdRegistryValue -Path $intlPath -Name 'sMonDecimalSep' -Value '.'
    Set-WcdDecimalThreadCulture -DecimalSeparator '.' -ThousandsSeparator ','

    Write-WcdLog -Path $resolvedLogPath -Level 'INFO' `
        -Message 'Regional: decimal and currency separators forced to a period.'

    Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
        -ModuleName $moduleName -StepKey 'DecimalAndCurrency' `
        -Event 'Finish' -Kind 'success'

    # ------------------------------------------------------------ ZONE 5
    return [pscustomobject]@{
        Step    = 'DecimalAndCurrency'
        Success = $true
        LogPath = $resolvedLogPath
        Error   = ''
    }

} catch {
    # Something went wrong: note it, name the fix, and carry on
    $note = $_.Exception.Message
    $remedyKey = 'RegistryWriteFailed'
    if ($note -match 'access is denied|unauthorized') {
        $note = 'Registry key locked by GPO or access denied.'
        $remedyKey = 'RegistryGpo'
    }

    Write-WcdLog -Path $resolvedLogPath -Level 'ERROR' -Message "Regional: $note"
    Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
        -ModuleName $moduleName -StepKey 'DecimalAndCurrency' `
        -Event 'Finish' -Kind 'error'

    return [pscustomobject]@{
        Step      = 'DecimalAndCurrency'
        Success   = $false
        LogPath   = $resolvedLogPath
        Error     = $note          # the raw detail, for the log
        RemedyKey = $remedyKey     # what the technician should do
```

```
        }
    }
}
```

> **TIP**
> Understand this module and you understand all eight. `Config-Power` has six steps instead of one and `Config-Language` calls different cmdlets, but the shape never changes.

6.4 A word about `RemedyKey`

Notice what the `catch` block returns. It keeps the raw error **and** names a remedy — but it does not write the remediation sentence itself.

That is deliberate. The sentence is user-facing text, so it lives in the translation tables in `Invoke-WcdConfiguration.ps1`, in both languages, next to every other string a technician reads. The module only says **which** problem this is; the orchestrator decides how to say it, and in what language.

CHAPTER 7

Adding a new module

7.1 First: does it need to be a module at all?

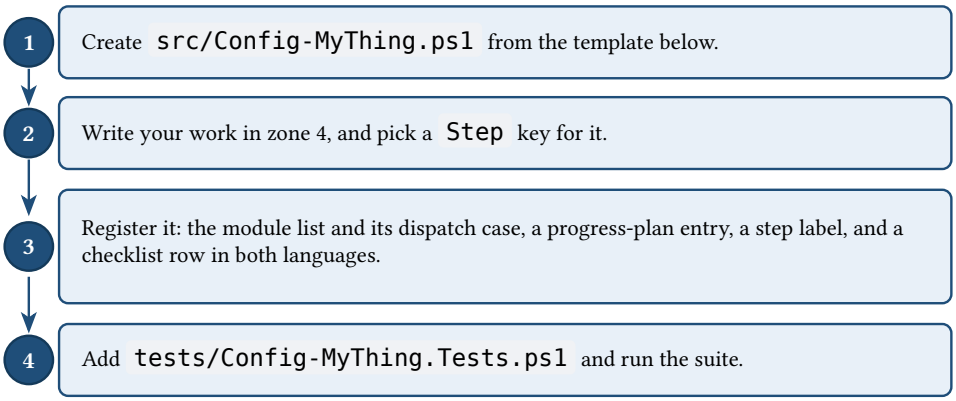
Most of what a technician wants to add is **opening or checking an application**, and that needs no code at all — it is one entry in `WinContextDeploy.psd1`:

```
@{
    Step   = 'AppTimeTracking'
    Name   = 'Time tracking portal'
    Action = 'OpenUrl'
    Target = 'https://time.example.com/'
}
```

That entry gets a progress bar, a checklist row, a log line, an entry in the JSON report and a remediation sentence when it fails — for free, because `Config-Applications` already does all of that for every entry in the list.

> **CAUTION**
> Write a module only when the manifest genuinely cannot express what you need: a registry change, a Windows setting, an inventory check. If the answer is "open this" or "check this exists", it is a manifest edit and you are done.

7.2 The four steps



7.3 The template

Copy this into `src/Config-MyThing.ps1` and replace `MyThing` and `MyStep` with your own names.

```
# Config-MyThing.ps1 - one line saying what this module is for.
# Entry point: Set-WcdMyThingConfiguration. Requires WcdHelpers.ps1.

function Set-WcdMyThingConfiguration {
    <#
    .SYNOPSIS
        One sentence: what this module does.

    .DESCRIPTION
        A paragraph: how it does it, and anything a reader would otherwise have
```

```
        to work out from the code - what needs Administrator, what is deliberately
        left out, what is safe to run twice.

    .PARAMETER LogPath
        Full path to the log file. Resolved automatically when omitted.

    .PARAMETER ProgressCallback
        Scriptblock invoked at the start and end of each step.

    .OUTPUTS
        [pscustomobject[]] with Step, Success, Error and, on a failure, RemedyKey.

    .EXAMPLE
        Set-WcdMyThingConfiguration -LogPath 'C:\temp\log.txt'
    #>
    [CmdletBinding()]
    param(
        [string]$LogPath,
        [scriptblock]$ProgressCallback
    )

    $resolvedLogPath = Resolve-WcdLogPath -CandidatePath $LogPath
    $moduleName = 'Config-MyThing'
    $results = @()

    try {
        Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
            -ModuleName $moduleName -StepKey 'MyStep' -Event 'Start'

        # ---------------------------------------------------------------
        # YOUR WORK GOES HERE
        # ---------------------------------------------------------------

        Write-WcdLog -Path $resolvedLogPath -Level 'INFO' -Message 'MyThing: done.'
        Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
            -ModuleName $moduleName -StepKey 'MyStep' -Event 'Finish' -Kind 'success'
        $results += [pscustomobject]@{ Step = 'MyStep'; Success = $true; Error = '' }
    } catch {
        Write-WcdLog -Path $resolvedLogPath -Level 'ERROR' `
            -Message ('MyThing: {0}' -f $_.Exception.Message)
        Invoke-WcdProgressCallback -ProgressCallback $ProgressCallback `
            -ModuleName $moduleName -StepKey 'MyStep' -Event 'Finish' -Kind 'error'
        $results += [pscustomobject]@{
            Step      = 'MyStep'
            Success   = $false
            Error     = $_.Exception.Message
            RemedyKey = 'MyStepFailed'   # add the sentence to both $T tables
        }
    }

    return $results
}
```

TIP

The template raises its own `Finish` event because it has one step. A module with several — `Config-Disk`

and `Config-Network` both have — should call `Complete-WcdProgressStep` instead: it reads the Result your module just recorded and picks the right progress kind, so a dozen branches never repeat the mapping.

```
Complete-WcdProgressStep -ProgressCallback $ProgressCallback `
    -ModuleName $moduleName -StepKey 'MyStep' -Results $results
```

7.4 Registering it

Four small edits, all in files you already have.

In `src/Invoke-WcdConfiguration.ps1` — add it to the module list and give it a dispatch case:

```
$modules = @(
    # ... the existing ones ...
    @{ Name = 'Config-MyThing'; File = 'Config-MyThing.ps1' }
)

# and, in the switch:
'Config-MyThing' {
    $modResults = @(Set-WcdMyThingConfiguration -LogPath $resolvedLogPath `
        -ProgressCallback $progressCallback)
}
```

In `src/WcdHelpers.ps1` — tell the progress bar how many steps to expect, and give the step a readable name:

```
# in Get-WcdModuleProgressPlan
'Config-MyThing' = @('MyStep')

# in Get-WcdTechnicalStepLabels
'MyStep' = 'My thing'
```

Back in `src/Invoke-WcdConfiguration.ps1` — add a checklist row and its label in **both** translation tables:

```
# in both $T tables, inside Checklist
MyThing = 'My thing'          # 'Mon truc' in the French table

# in Get-WcdFinalChecklistEntries
$entries += Resolve-WcdAutomaticEntry -Label $T.Checklist.MyThing `
    -ResultLookup $lookup -StepKeys @('MyStep') -StepLabels $StepLabels
```

7.5 Testing it

Every module has a test file, and yours should too. The pattern is always the same: dot-source the helpers and the module, mock whatever touches the machine, and assert on the Results.

```
Describe 'Config-MyThing' {
    BeforeAll {
        $srcDir = Join-Path (Split-Path $PSScriptRoot -Parent) 'src'
        . (Join-Path $srcDir 'WcdHelpers.ps1')
        . (Join-Path $srcDir 'Config-MyThing.ps1')
    }

    It 'succeeds when the work goes through' {
```

```
        Mock -CommandName 'Set-WcdRegistryValue' {}

        $results = @(Set-WcdMyThingConfiguration -LogPath (Join-Path $TestDrive
'log.txt'))

        $results[0].Step | Should -Be 'MyStep'
        $results[0].Success | Should -BeTrue
    }

    It 'reports a failure without throwing' {
        Mock -CommandName 'Set-WcdRegistryValue' { throw 'Access is denied' }

        $results = @(Set-WcdMyThingConfiguration -LogPath (Join-Path $TestDrive
'log.txt'))

        $results[0].Success | Should -BeFalse
        $results[0].Error | Should -Match 'denied'
    }
}
```

TIP

Mock anything that touches the real machine — the registry, `powercfg`, `Add-Printer`, `Start-Process`. A test that changes the machine it runs on is a test nobody will run twice.

Run the suite with `Invoke-Pester .\tests\*.Tests.ps1 -Output Detailed`, or by double-clicking `Lancer-Tests-Pester.cmd`.

CHAPTER 8

Troubleshooting

8.1 Reading the diagnostic statuses

Every row of the final checklist carries one of five statuses. Telling them apart is most of what troubleshooting is.

Status	Colour	What it means
`[x] OK`	Green	The step did what it was supposed to. It may still carry a note — an Optional application that is simply not installed says so here.
`[!] WARNING`	Yellow	Something worth your attention that is not a failure: a VPN client not running, a print server out of reach, a power step that needed Administrator.
`[!] ERROR`	Red	The step should have worked and did not. Read the remediation on the same line; the raw detail is in the log.
`[-] MANUAL`	Yellow	Deliberately not automated. Yours to do — see chapter 4.
`[-] N/A`	Grey	Does not apply to this machine's form factor or environment. Its absence is correct, not a problem.

> **TIP**
> The line under a warning or an error is the one to act on. It names the fix, and where a manifest entry is at fault, the exact key to edit:

```
  [!]  ERP client            ERROR    ERP client not found at C:\ProgramData\...
                                      -> Update Applications['ERP client'].Target in
                                         WinContextDeploy.psd1, or remove the entry.
```

8.2 Reading the logs

Three files can come out of a run, and they answer different questions.

The run log `src/log.txt`	Everything this run did, line by line, with the **raw** error text. Override the location with `-LogPath`. This is where you look when the remediation on screen is not enough.
The history log `-HistoryLogPath`	One block appended per machine — name, serial number, model, user, the whole run log and the final diagnostic. One file becomes the record of every machine you configured. The `-Usb` launcher points this at the key automatically.
The JSON report `-ReportPath`	The same result, machine-readable, for collecting across a fleet. Carries a `schemaVersion` so a collector can version against it.

Each log line is a timestamp, a level, and a message:

```
2026-08-29 09:14:23 [INFO]  Power: screen timeout on AC set to 15 min.
2026-08-29 09:14:23 [INFO]  Power: active scheme applied.
2026-08-29 09:14:24 [INFO]  Regional: decimal and currency separators forced to a period.
2026-08-29 09:14:25 [ERROR] Taskbar align: Registry key locked by GPO or access denied.
2026-08-29 09:14:26 [INFO]  Display language: fr-CA applied.
2026-08-29 09:14:27 [WARNING] Set-WinSystemLocale not applied (Access denied).
```

> **NOTE**
> An `[ERROR]` line does not mean the tool crashed. It never stops on a failed step: the error is recorded so you can deal with it afterwards, and the run continues to the end. A report you can read beats a run that stopped at step three.

8.3 Common problems

Symptom	Likely cause	What to do
The window closes immediately	PowerShell execution policy	Launch through `WinContextDeploy.cmd`, never the `.ps1` directly. The launcher passes `-ExecutionPolicy Bypass` for exactly this reason.
Every power step says it needs Administrator	You declined the UAC prompt, or the account has no rights to give	Nothing is broken — everything else applied. Relaunch elevated to finish the power settings, or accept them as they are.
`Registry key locked by <name> in GPO` the log	Group Policy owns that setting	Expected on a managed machine. The change has to be made in Group Policy; the tool cannot and should not fight it.
An application says it was not found	The manifest points at a path that does not exist here	The remediation names the key: fix `Applications['<name>'].Target`, or remove the entry if that application is not part of this image.
`Module not found` in the diagnostic	A `Config-*.ps1` is missing or misnamed	Check that the file exists in `src/` and that its name matches the `File` value in `$modules`.
`Free space` warns on a machine that is fine	The threshold does not suit this fleet	The remediation names the key: raise or lower `Disk.MinFreeGB` in the manifest. It defaults to 20 GB, which is generous on a 128 GB endpoint.
The connectivity test fails on a working network	The network drops ICMP to the internet	Point `Network.PingTarget` in the manifest at your gateway or an internal host.
The history export failed after elevating	`-HistoryLogPath` is on a network share	UAC opens a new logon session, which drops mapped drives. Use a local or USB path, or run elevated from the start. The tool warns before elevating when it sees a UNC path.
A printer never connects	The print server is unreachable, or the queue name is wrong	Check `Printers[].Connection` against what the print server publishes. `Name` must match the published queue name for the idempotency check to work.

8.4 Getting more detail

Every function in `src/` carries proper help. When you want to know what something does or what it returns, ask PowerShell rather than reading the source:

```
Get-Help Set-WcdPowerConfiguration -Full
Get-Help Get-WcdApplicationTarget -Examples
```

And when a change to the code is what you need, the test suite is the fastest way to find out whether it worked:

```
Invoke-Pester .\tests\*.Tests.ps1 -Output Detailed
```